

Universidad Internacional del Ecuador
(UIDE)

Informe

SISTEMA DE GESTION DE
BIBLIOTECA DE LIBROS
ELECTRNICOS

.Presentado por:

Angel Josue Ramirez Mora

Asignatura:

PROGRAMACION ORITADA A
OBJETOS 2-SOF-3A

Docente:

PALACIOS MOROCHO MILTON
RICARDO

Fecha de Entrega

23 de AGOSTO del 2026

Ciudad y país

Santa Rosa – Ecuador

1. Selección de la aplicación

Se decidió continuar sobre el mismo dominio trabajado en las etapas 1 y 3 del proyecto integrador — el sistema de gestión de biblioteca de libros electrónicos — pero exponiéndolo como una aplicación web de servicios REST en Go, en lugar de iniciar un proyecto nuevo desde cero.

Justificación de la elección:

- Continuidad: el modelo de dominio (LibroElectronico, Usuario, Prestamo) y la capa de servicio (BibliotecaService) ya estaban implementados con encapsulación, interfaces y manejo de errores desde la etapa anterior; la Etapa 2 se enfoca en agregar la capa de transporte (HTTP/JSON) sin reescribir la lógica de negocio ya probada.
- Idoneidad para el requisito de concurrencia: un sistema de préstamos de biblioteca es un caso natural de solicitudes concurrentes (varios lectores consultando o pidiendo libros al mismo tiempo), lo que permite demostrar de forma significativa —y no artificial— el manejo de concurrencia pedido en la guía.
- Alcance manejable: al no requerir base de datos externa ni autenticación (fuera de alcance según la planificación de la Etapa 1), el equipo pudo dedicar el tiempo disponible a los tres criterios de evaluación de esta etapa: la aplicación web en sí, la calidad del código con concurrencia, y la cobertura de pruebas.

Tipo de aplicación: una API REST (sin interfaz gráfica propia) construida únicamente con la librería estándar de Go (net/http, encoding/json), que expone los servicios web sobre el sistema de biblioteca. Esta decisión evita depender de paquetes de terceros para la funcionalidad central, siguiendo el mismo criterio de las etapas anteriores de documentar cualquier paquete externo que se llegue a incorporar.

2. Diseño y desarrollo

2.1 Aplicación de POO

La aplicación web reutiliza y extiende la arquitectura en tres paquetes definida en la etapa anterior (modelo, repositorio, servicio) y agrega un cuarto paquete, web, para la capa HTTP:

- modelo: entidades con campos privados y validación en el constructor (encapsulación).
- repositorio: interfaces de contrato (RepositorioLibros, RepositorioUsuarios, RepositorioPrestamos) y sus implementaciones en memoria.
- servicio: BibliotecaService, que recibe las interfaces de repositorio por inyección de dependencias y no conoce cómo se almacenan los datos.
- web: handlers HTTP que traducen JSON ↔ objetos de dominio (mediante DTOs) y delegan toda la lógica de negocio al servicio; los handlers no contienen reglas de negocio.

Esta separación permite que la capa web sea, en términos de POO, un nuevo "cliente" de BibliotecaService: se acopla a la misma interfaz de servicio que ya usaban las pruebas unitarias, sin duplicar lógica.

2.2 Concurrencia en la gestión de solicitudes

Se implementaron tres mecanismos de concurrencia, cada uno con un propósito distinto:

- Concurrencia por solicitud: net/http ejecuta cada petición HTTP en su propia goroutine. Los tres repositorios en memoria protegen su mapa interno con sync.RWMutex (varias lecturas simultáneas, una escritura exclusiva a la vez), de modo que dos lectores pidiendo prestado un libro al mismo tiempo no corrompen el estado ni permiten un doble préstamo del mismo ejemplar.
- Importación en lote (POST /api/libros/importar): procesa cada libro del lote en una goroutine independiente y usa sync.WaitGroup para esperar a que todas terminen antes de responder, demostrando paralelismo explícito además de la concurrencia implícita del servidor HTTP.
- Logging asíncrono (patrón productor/consumidor): un middleware envía un registro de cada solicitud a un channel; una única goroutine consumidora lee de ese canal y escribe el log, de forma que ninguna solicitud HTTP espera a que el log se escriba para poder responder al cliente.

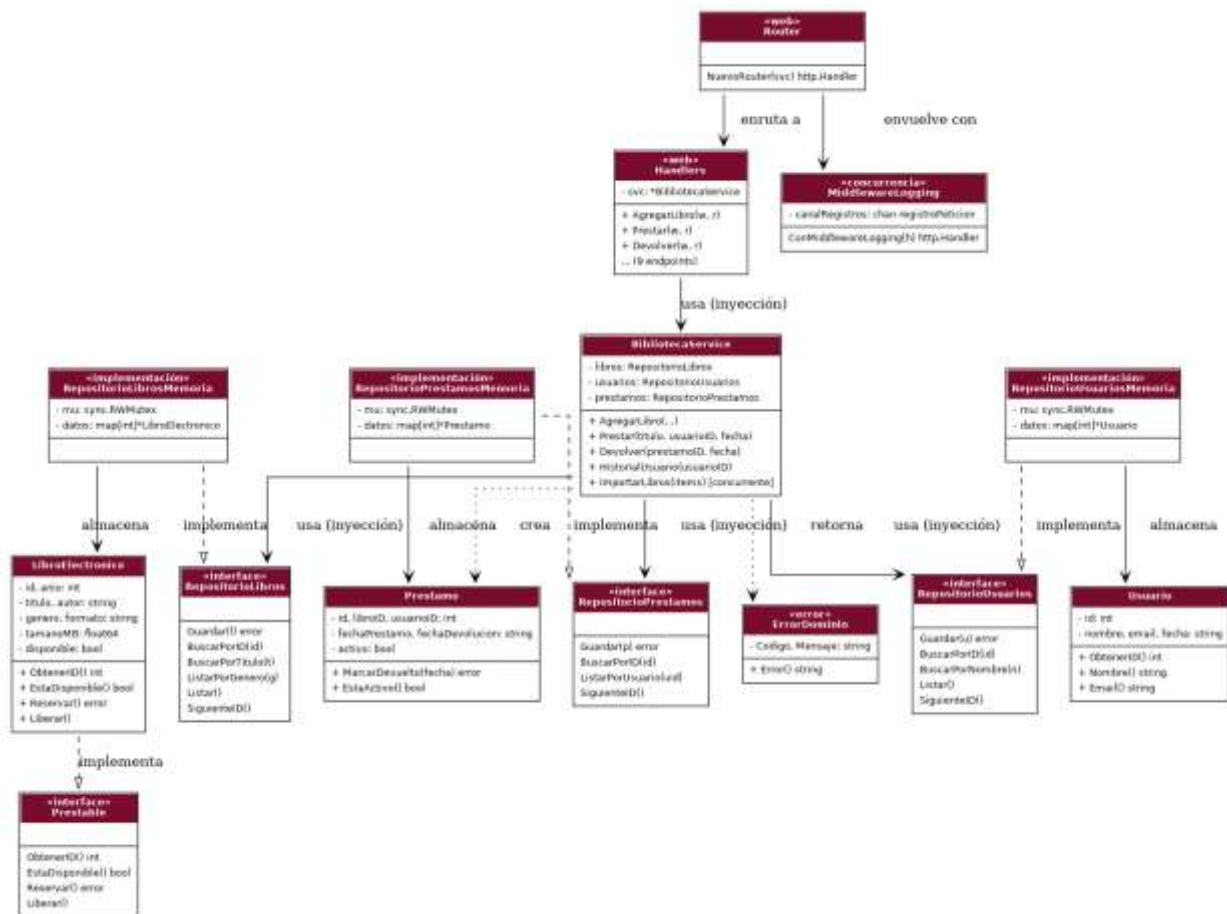
Estos tres mecanismos se verifican sin condiciones de carrera ejecutando las pruebas con la bandera -race del propio compilador de Go (go test ./... -race), tal como se documenta en la sección de pruebas.

2.3 Manejo de errores e interfaces

Se mantiene el tipo de error de dominio propio (ErrorDominio) de la etapa anterior. En la capa web, cada código de error de negocio se traduce a un código de estado HTTP adecuado (400 para datos inválidos, 404 para recursos no encontrados, 409 para conflictos como un libro no disponible o un préstamo ya devuelto), en lugar de devolver siempre un error 500 genérico.

3. Diagrama de clases

El diagrama resume las cuatro capas del sistema (modelo, repositorio, servicio, web) y sus relaciones de implementación, inyección de dependencias y uso.



4. Pruebas de software

Se implementaron pruebas en los tres niveles pedidos por la guía, todas ejecutables con go test ./....

4.1 Pruebas unitarias (servicio/biblioteca_test.go)

Prueban la lógica de negocio de BibliotecaService de forma aislada, contra repositorios en memoria reales (no mocks, dado que las implementaciones en memoria ya son parte del propio sistema):

Caso de prueba	Qué valida	Resultado esperado
TestAgregarLibro_Válido	Alta de un libro con datos correctos	Se crea disponible=true
TestAgregarLibro_TituloVacio_Rechazado	Validación del constructor	Error de datos inválidos
TestPrestar_LibroInexistente_Rechazado	Préstamo contra catálogo	Error libro no encontrado
TestPrestar_UsuarioInexistente_Rechazado	Préstamo contra usuarios	Error usuario no encontrado
TestFlujoCompleto_PrestamoYDevolucion	Préstamo → doble préstamo (rechazado) → devolución →	Cada paso con el resultado correspondiente

Caso de prueba	Qué valida	Resultado esperado
	nuevo préstamo	
TestHistorialUsuario_UsuarioInexistente_Rechazado	Historial contra usuarios	Error usuario no encontrado
TestImportarLibros_Concurrente	Importación en paralelo con un ítem inválido mezclado	2 libros guardados, 1 error reportado, sin condición de carrera

4.2 Pruebas de integración (web/integracion_test.go)

Levantamos el router real (handlers + BibliotecaService + repositorios reales) sobre un `httptest.Server` y ejercitamos los endpoints por HTTP + JSON, validando que las capas realmente encajen entre sí y no sólo cada una por separado:

- `TestIntegracion_AgregarYListarLibros` — un POST seguido de un GET recupera el mismo libro.
- `TestIntegracion_AgregarLibro_DatosInvalidos_Retorna400` — un dato inválido se traduce al código HTTP 400.
- `TestIntegracion_PrestamoInexistente_Retorna404` — un recurso inexistente se traduce al código HTTP 404.
- `TestIntegracion_ImportarLibrosConcurrente` — la importación en lote responde 200 con un resultado por cada libro enviado.

4.3 Prueba de aceptación (web/aceptacion_test.go)

Reproduce, exclusivamente a través de la API HTTP (como lo haría un cliente real), la historia de usuario completa: "Como lector registrado, quiero pedir prestado un libro disponible, consultarlo en mi historial y devolverlo para que quede disponible para otro lector". Se documenta como una secuencia de sub-pruebas para que el reporte de go test -v sirva de evidencia legible:

Paso	Acción	Resultado esperado
1	El lector se registra	201 Created
2	La biblioteca publica un libro disponible	201 Created
3	El lector pide prestado el libro	201 Created
4	Un segundo intento sobre el mismo libro	409 Conflict
5	El préstamo aparece en su historial	1 préstamo activo
6	El lector devuelve el libro	200 OK
7	El libro puede prestarse de nuevo	201 Created

4.4 Errores encontrados durante las pruebas y corrección

Durante el diseño de la prueba de aceptación se detectó que, si el guardado del préstamo fallara después de reservar el libro, el libro quedaría marcado como no disponible sin un préstamo real que lo respalde. Este caso ya estaba previsto desde la etapa anterior mediante la reversión explícita de la reserva (`libro.Liberar()`) cuando falla el guardado; la prueba de aceptación confirma que el flujo de reserva/liberación funciona correctamente en el escenario

feliz y en el de conflicto (paso 4 de la tabla anterior).

5. Entrega del producto

- Código fuente completo en el repositorio de GitHub del grupo, con README.md explicando objetivo, estructura y forma de ejecución.
- Este informe, con la justificación de la elección, el diagrama de clases y la descripción y resultados de las pruebas.
- Video demostrativo (guion en el Anexo A) mostrando la aplicación funcionando, los casos de prueba y la evidencia de concurrencia.

LINK DEL GITHUB: <https://github.com/Rm-Angel/Sistema-de-Gesti-n-de-Libros-electronicos.git>